Environment-Aware Malware

EVALUATION OF COMMON ENVIRONMENT AWARENESS TECHNIQUES USED TO HIDE AND WEAPONISE MALWARE & HOW THEY CAN BE MITIGATED

Zack Anderson (1602117@uad.ac.uk) | BSc Ethical Hacking | 07/05/2019

Abstract

This paper discusses research of the techniques most commonly used by malware to analyse its environment after infecting a system. Environment awareness can be used by malware to evade detection by users or automated analysis environments before reaching its target. 3 of the most successful samples of malware in recent years; *WannaCry, Kovter,* and *ZeuS,* were analysed to determine how they detect their environment.

To conclude, this paper discusses how companies can defend themselves and minimise the impact of cyber-attacks by hardening sandbox analysis implementations and by utilising cloud tools and services to virtualise end-point desktop environments.

Contents

Introduction 2
Common Environmental Detection Techniques 3
Environment Checking 3
Stalling 3
Interaction 4
Fingerprinting 4
IsDebuggerPresent 4
Practical Analysis 5
Tools Used 5
Setup 5
Installing Cuckoo 5
Methodology 7
Research & Results 8
WannaCry 8
Kovter 12
ZeuS 14
Discussion 17
Hardening Sandboxes 17
Virtualisation of Desktop Environments 17
Future Work 18
References 19
Appendices 21

Introduction

As cyber-attacks grow in both scope and complexity – it is becoming ever more difficult to respond swiftly to an attack on a computer system. When evaluating new threats, cyber-security specialists implement an automated solution known as a "*sandbox*" – a form of a virtual machine which is isolated and monitored throughout the execution of the application – to determine if the new application is malicious.

The response from attackers was to develop a new form of environmentally aware malware which tests various aspects of the system it is installed on to determine if it is being executed within these "*sandboxes*". This is known as "*sandbox aware*" or "*sandbox evading*" malware. These types of malware will either not run if they suspect they are in a sandbox (a solution which would raise suspicion in most modern automated solutions), or they will carry out a mundane task to masquerade as a normal piece of software and bypass these automated checks. Sandbox aware malware is a rapidly growing issue – being placed 2nd on *CSOOnline*'s "*7 cybersecurity trends to watch out for in 2018*". [1]

Back in July of 2010, reports of a new, incredibly aggressive malware started coming in from all over the world. This malware was like nothing seen before – utilising multiple zero-day vulnerabilities (vulnerabilities that were previously not found and have no patch), combined with immaculate code which was so complex it took industry experts months to even begin to understand its functions. This malware was known as "*Stuxnet*", another example of environmentally aware malware.

The *Stuxnet* malware used its environment awareness to lay dormant to avoid detection until it was installed on computers connected to a specific type of programmable logic controller (*PLC*). This PLC was specifically used by the Iranian government when controlling the enrichment of Uranium. Once the malware was installed, it would gain control of the PLC and start recording data. After roughly 2 weeks the malware would begin to manipulate the PLC to damage centrifuges while reporting back data it had recorded previously and disabling the emergency shutdown – resulting in centrifuges self-destructing. [2]

Stuxnet is an example of malware which uses its environmental awareness to hide from automated checks until it is in the right environment, which is when it weaponizes itself and causes a serious threat to life.

This paper will discuss the most commonly used environment detection techniques, followed by a practical analysis of 3 pieces of malware and the environment detection techniques they implement.

Common Environmental Detection Techniques

Environment Checking

Malware can try to identify the environment it is running in by checking various aspects of its environment – for example, scanning for common registry entries or files created by sandboxing or virtual machine software. Other ways this can be achieved include:

- Checking for internet access.
- Specifying a date and time the code should be executed at.
- Checking the number of CPU cycles between 2 instructions executed right after each other (which would be much higher in a virtual machine).
- Scanning for commonly installed user software such as web browsers and PDF viewers.
- Scanning for common files created by the virtual machine software.
- Activating after a reboot or shutdown of the environment (which is usually done by writing to the Master Boot Record of the device). [3]
- Checking for system uptime.

In response to this, sandboxes will often fake internet access by responding to all requests or by changing the system clocks on the victim virtual machine – however, more complex variations of malware will detect these changes and continue to avoid detection.

Environment checking can also be used to weaponize malware as it may be possible for the malware to determine if it is on a specific network before it starts running malicious code. This allows it to sit undetected on *"carrier"* computers until they are plugged into a network where the malware can spread. This was used as part of the *"Stuxnet"* attack, where the attackers targeted maintenance companies to carry the malware inside the network used by the Iranian nuclear program which was isolated from the internet.

Stalling

One of the simplest methods of determining whether an application is in a sandbox is *"stalling"* the code. *Stalling* the code involves fooling automated checking by waiting a certain amount of time before executing malicious code.

This method is usually effective because larger company-wide sandboxes may have tens or even hundreds of samples to analyse – meaning it can only dedicate a certain amount of time to each sample. Some analysis software will adjust the system time to make the sample believe that the duration for which the process was meant to sleep has passed.

Malware developers have become aware of this and will often compare the time passed with system uptime by checking the number of CPU cycles passed or using "*GetTickCount*" – which calculates system uptime in milliseconds – before and after the instruction was executed which would show a discrepancy in time passed when compared to the system uptime. Some malware samples will perform a set of ordinary looking tasks designed to waste CPU cycles rather than just sleeping with the intention of making any automated sandbox testing time out.

Interaction

Another solution way for an application to determine if it is in a sandbox which is commonly used by malware developers is to wait for human interaction. Approaches to this can include waiting for the user to type a certain number of keys, click a certain number of times, or scroll a certain amount of lines before executing. Some samples will even look for suspiciously fast or algorithmic interactions which would suggest automated input.

This technique could be mitigated by recording sample user inputs for the sandbox to emulate during its analysis.

Fingerprinting

Some strains of malware will generate a unique hash or fingerprint when they arrive at a destination computer. When executed, they will generate this hash again and compare it to the original. If the hashes do not match, the malware will know it is not in the original destination environment and therefore will not execute its malicious code.

To bypass this measure, a sandbox would have to find a way to get the hash from the original computer or manipulate the program to think the hashes matched.

IsDebuggerPresent

"*IsDebuggerPresent*" is a function found in the "*kernel32*" library in all modern Windows operating systems which will return a value if a user-mode debugger is present for the process. [5] If the malware identifies a debugger – it can either prevent the execution of its malicious code in the ways discussed above, or it can attempt to attack and crash the debugger. This can be patched out by:

- Modifying the function in the library.
- By modifying the function call to *isDebuggerPresent* in a hex-editor and replacing it with "*No Operation*" calls.
- During execution by replacing the call with "*No Operation*" calls within memory during execution.
- Or by placing a breakpoint after the call and modifying the register values to show that no debugger was present. [6]

Practical Analysis

To better understand how malware carries out environment checks, an analysis of 3 environment-aware malware samples was carried out. The 3 samples chosen were *WannaCry, Kovter,* and *ZeuS.* These were successful in infecting computers worldwide due to their implementations of environment detection. These samples were downloaded from a GitHub repository for malware known as "*theZoo*". [8]

TOOLS USED

- **Cuckoo Sandbox** – an open-source sandboxing software used to analyse applications.
- **VirtualBox** – open-source virtual machines and networks manager.
- **Ghidra** – an open-source tool used for static analysis of applications.
- **Git** – a version control system, used to clone GitHub repositories.
- **Vim** – a command line text editor.
- **Python** – a scripting language.

SETUP

Installing Cuckoo

The first step was to install Cuckoo on a fresh installation of Ubuntu 18.04. Using a tutorial by *Nicholas Dionysopoulos,* a fully portable install of Ubuntu can be created on an external USB hard drive. [9] This means the primary hard drive of the computer can be unmounted to protect it in the unlikely event of malware escaping the sandbox.

A 64-bit virtual machine called "*cuckoo1*" running Windows 7 Ultimate was pre-configured using VirtualBox and used as the "*victim*" machine in this case, with "*User Account Control*" and the Windows Firewall disabled to make sure malware execution was not prevented by Windows. *Python* is required to be installed on the client to run the Cuckoo "*agent.py*" script, which communicates with the *"result server"* in the host machine to construct a report throughout the execution of the sample.

A user-created script called "*cuckoo-install.sh*" was used to install Cuckoo, which can be found on GitHub at https://github.com/NVISO-BE/SEC599/blob/master/cuckoo-install.sh. This was edited using "*vim*" to pre-configure Cuckoo to run with the Windows 7 virtual machine. These edits can be found in *Appendices – "cuckoo-install.sh" Configuration.*

One issue that arose while installing Cuckoo on Ubuntu 18.04 is that compatibility with the Python tool "*pip*" was broken in the latest update. One way to work around this would be to use an older version of Ubuntu such as 16.04, or to find what components were not installed using the "*--verbose*" instruction when running "*cuckoo-install.sh*" and manually install them using the "*apt install*" command.

A virtual network can then be created and isolated from other networks on the host computer using the commands seen in *Appendices – Network Configuration Commands.* The *"resultserver"* and Windows 7 virtual machine should both run on this network, which can be achieved by editing the *"cuckoo.conf"* file.

The *"agent.py"* script was then copied from the host to the virtual machine and executed with administrator privileges, and then saved to a snapshot called *"clean1"*. The name of this snapshot was then added to the configuration files of Cuckoo in the *"cuckoo.conf"* so that Cuckoo knows which snapshot to load before each run.

Running the command *"cuckoo -d"* should start the Cuckoo services in debug mode to allow the user to troubleshoot any errors or issues they encounter.

```
2019-05-03 13:15:09,746 [cuckoo.core.startup] DEBUG:     |-- office ole.yar
2019-05-03 13:15:09,747 [cuckoo.core.resultserver] DEBUG: ResultServer running on 192.168.56.1:2042.
2019-05-03 13:15:09,748 [cuckoo.core.scheduler] INFO: Using "virtualbox" as machine manager
2019-05-03 13:15:10,674 [cuckoo.machinery.virtualbox] DEBUG: Stopping vm cuckoo1
2019-05-03 13:15:10,958 [cuckoo.machinery.virtualbox] DEBUG: Restoring virtual machine cuckoo1 to clean1
2019-05-03 13:15:11,387 [cuckoo.core.scheduler] INFO: Loaded 1 machine/s
2019-05-03 13:15:11,401 [cuckoo.core.scheduler] INFO: Waiting for analysis tasks.
```

Figure 1 – Cuckoo terminal output when booted successfully

To submit a file for analysis, the front-end web interface needs to be booted using *"cuckoo web"* in a separate terminal window. Once the server has loaded, it can be accessed by default on *"localhost:8000"*.

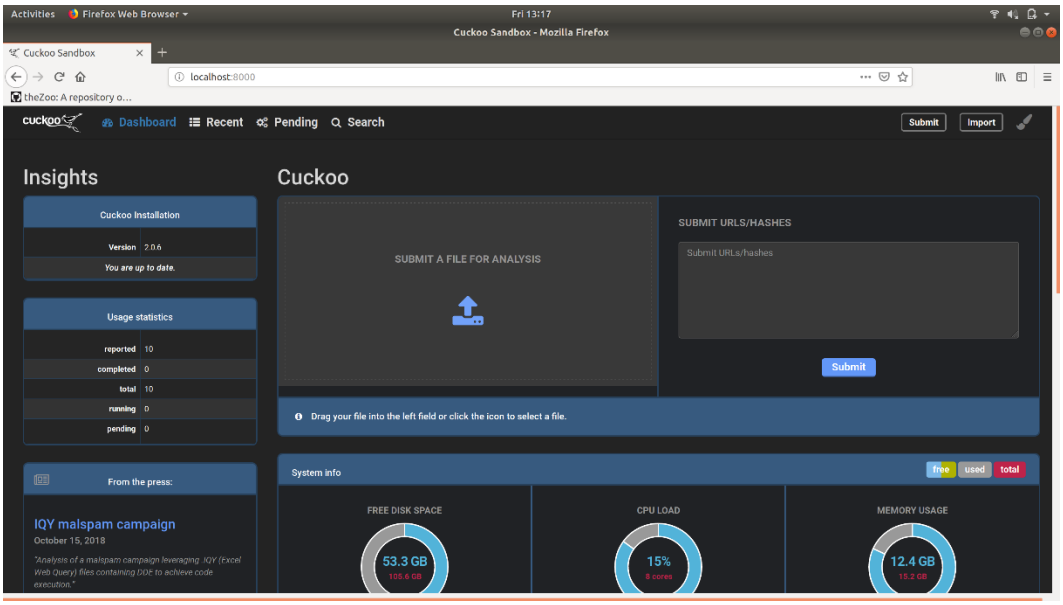


Figure 2 – Cuckoo web interface

Malware samples can then be submitted for analysis using the web interface. *Ghidra* can also be used to statically analyse the malware to further analyse the results found by *Cuckoo.*

METHODOLOGY

The first step of the methodology was to choose a sample or samples of malware to be analysed and uploading them to Cuckoo using the web interface.

Once the sample has been analysed, *Cuckoo* will generate an in-depth report on the program which will includes:

- A summary of the analysis, scoring the program on its suspicious functions, complete with screenshots.
- An analysis of the network traffic generated by the process
- A memory dump from the process
- A list of files created, deleted and altered by the process
- A list of all library functions called by the application compiled during automated static analysis of the sample

Research & Results

WannaCry

WannaCry is a form of malware which encrypts users' files on the computer then demands a payment in order to decrypt them. This type of malware is known as *"ransomware"*. In this case the ransom was a $300 payment of Bitcoin into a wallet address given to the victim in the message displayed once their files were encrypted. In the case of *WannaCry*, it enumerates the files it has encrypted and offers a *"demo decryption"* feature which decrypts files it deems unimportant to try and get the victim to pay the amount. However, analysis of the malware has identified it does not include a way to decrypt the rest of the files even if the amount is paid. [10]

Figure 3 – The screen displayed once WannaCry encrypts a victims' hard drive

This is why, among other reasons, it is recommended that users' back-up important files regularly.

The *WannaCry* ransomware became infamous after infecting government systems and critical infrastructure around the world. In the UK, the NHS systems were hit hard due to out-of-date Windows environments, costing them an estimated £92 million in total and resulting in over 19,000 appointments being cancelled. [11]

WannaCry used *"EternalBlue"* – a vulnerability in Microsoft's implementation of the *"Server Message Block" (SMB)* protocol which allowed unauthorised access to machines to read and write files or request system services [11]. This allowed it to spread across networks and infect as many computers as possible. This vulnerability was found by the US National Security Agency (NSA) and was not disclosed by them as they planned to use it for their own exploits – and was then leaked onto the internet sometime in early 2017.

The patch for the vulnerability was released on March 14th, 2017, 2 months before the *WannaCry* malware made headlines in May [13] – and was flagged as critical by Microsoft who even went as far as to release an update for Windows XP 3 years after its support was dropped. However, many companies still did not update their software, including the NHS.

Environment Checks

WannaCry utilised a combination of environment checks to bypass sandbox analysis. In one version of the malware, one of these checks was checking for internet access by trying to query an unregistered URL. If a sandbox was trying to emulate internet access, the malware would get a reply from the server and shutdown. The malware was stopped by *Marcus Hutchins,* who found the URL and registered it to map infections of the malware, only to find he had accidentally stumbled across a way to kill the malware. [14]

The "Kill-Switch"

This kill switch is not found in the version of *WannaCry* found in *theZoo,* but to show its existence another variant of the malware, posted by *"Ghidra Ninja",* was downloaded and loaded into *Ghidra.* [15]

The first function of the malware is the *"entry"* function. As this malware is a Windows executable program, the function called at the end of this function must be the *"WinMain"* function. The definition for this can be found online in Microsoft's documentation [16] and imported into *Ghidra*, as shown below.

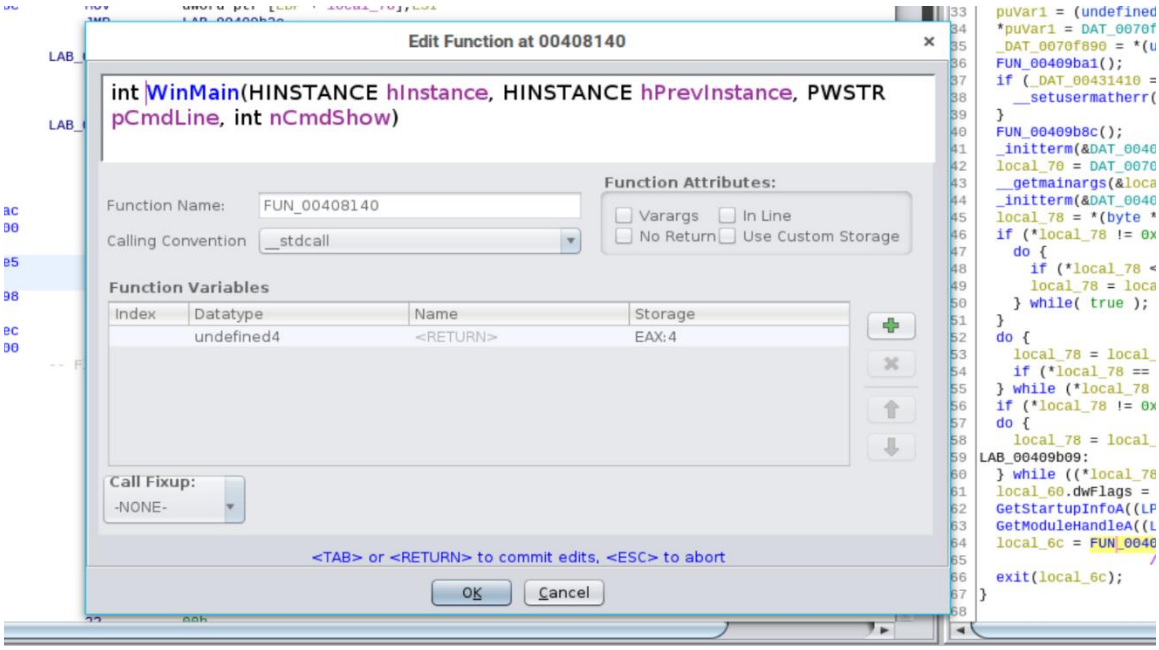


Figure 4 – Importing the *WinMain* definition into *Ghidra*

Once the function is defined, it can be viewed by double-clicking it in the de-compiler view. This can be seen in *Appendices – WinMain Function.* The first notable item in this function is a URL. To help understand what this function is doing, the variables can be renamed in the de-compiler view and their type defined as their suspected data type.

There is a *"while"* loop in this function that appears to push the URL to the stack in 4-byte segments, as seen below.

```
url_copy = url_buffer;
while (i != 0
                    /* strcopy(URL_copy, URL1, 14) */) {
  i = i + -1;
  *(undefined4 *)url_copy = *(undefined4 *)URL1;
  URL1 = URL1 + 4;
  url_copy = url_copy + 4;
}
*url_copy = *URL1;
```

Figure 5 – while loop pushing the URL to the copy in 4 byte segments

The *"InternetOpenA"* and *"InternetOpenUrlA"* functions are also standard Windows functions with definitions available in Microsofts documentation. However, when trying to import the *"InternetOpenUrlA"* function, an error appears as the variable type *"HINTERNET"* is not defined.

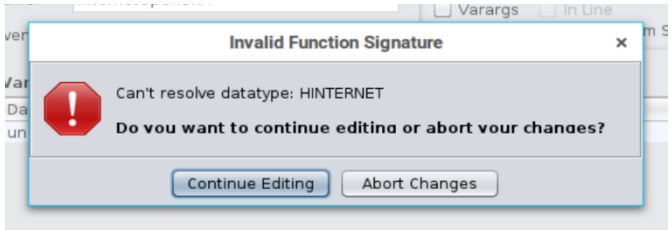


Figure 6 – Error when importing *"InternetOpenUrlA"*

This error can be fixed by defining the *"HINTERNET"* data type.

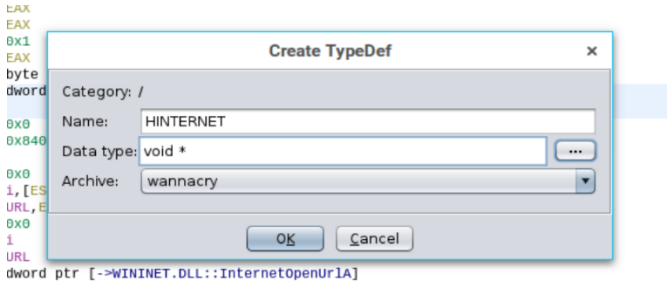


Figure 7 – Defining *HINTERNET*

This then allows us to define *"InternetOpenUrlA"* and we can see what the function is doing. It makes a request to the URL defined at the start of the function and if there is no reply then the *"hInternetReturn"* variable will be *null,* and the malware proceeds to call the program. If it does get a reply, it closes the handlers and returns 0, causing the application to close.

```
InternetOpenA((LPCSTR)0x0,1,(LPCSTR)0x0,(LPCSTR)0x0,0);
hInternetReturn = InternetOpenUrlA(hInternet,url_buffer,(LPCSTR)0x0,0,0x84000000,0);
if (hInternetReturn == (HINTERNET)0x0) {
  InternetCloseHandle(hInternet);
  InternetCloseHandle(0);
  real_entry();
  return 0;
}
                /* Tries to request the URL. If it fails, it closes the handler & calls a
                   function. If it succeeds, it closes the handler and returns 0, closing the
                   program */
InternetCloseHandle(hInternet);
InternetCloseHandle(hInternetReturn);
return 0;
```

Figure 8 – The original *"WannaCry kill-switch"*

Other Environment Checks

Moving on to the later version of *WannaCry* found in *theZoo,* this sample checks for sandboxing by using the *"IsDebuggerPresent"* function to test for the presence of a debugger on the process, as well as checking the amount of memory and storage on the system to possibly identify a virtual machine, which usually have small storage drives and low amounts of memory. It does this by calling the *"GlobalMemoryStatusEX"* and *"GetDiskFreeSpaceExW"* functions.

Checks if process is being debugged by a debugger (5 events)

Time & API
IsDebuggerPresent April 29, 2019, 7:41 p.m.

Figure 9 – WannaCry calling *IsDebuggerPresent*

Checks amount of memory in system, this can be used to detect virtual machines that have a low amount of memory available (1 event)

Time & API	Arguments	Status
GlobalMemoryStatusEx April 29, 2019, 7:41 p.m.		1

Queries the disk size which could be used to detect virtual machine with small fixed size or dynamic allocation (4 events)

Time & API	Arguments
GetDiskFreeSpaceExW April 29, 2019, 7:41 p.m.	total_number_of_free_bytes: 32170418176 free_bytes_available: 32170418176 root_path: C:\ total_number_of_bytes: 42842714112

Figure 10 – Function used by the sample to check the amount of memory and storage on the system

It also tries to sleep the process for 992 seconds to cause a timeout of any sandbox analysis programs and queries the system for the computer name – perhaps to try and identify a computer with a suspicious name such as "*malwareanalysis-PC*", as can be seen below.

Queries for the computername (10 events)

Time & API	Arguments
GetComputerNameW April 29, 2019, 7:41 p.m.	computer_name: ZACK-PC

Figure 11 – WannaCry checking the computer name

A process attempted to delay the analysis task. (1 event)

description	ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa.exe tried to sleep 992 seconds, actually delayed analysis time by 992 seconds

Figure 12 – WannaCry trying to sleep the function

It carries out further checks by searching the currently running processes in the machine, likely for virtualisation software processes, and searches for the "*agent.py*" file used by Cuckoo to communicate with the result server.

Attempts to detect Cuckoo Sandbox through the presence of a file (1 event)

file	C:\Users\Zack\AppData\Local\Temp\VirtualBox Dropped Files\2019-04-20T22_59_06.715060600Z\agent.py

Figure 13 – WannaCry trying to find the *agent.py* file

Kovter

"*Kovter*" is another example of ransomware. This sample was famous for its unconventional techniques for persistence and to bypassing sandboxes. It can be traced back to 2013 where it initially attempted to extort money from victims by claiming to be from a variety of law enforcement agencies and claiming they had found incriminating material on the victims' hard drive [18]. It later to become better at evading detection and, in 2015, the early versions of what became known as *Kovter* started to appear.

Kovter is a file-less application – meaning it does not write any files to the hard drive of the victim computer allowing it to remain mostly undetected. [19] It did this by creating registry entries that were autorun when a specific action was carried out. In 2016 it evolved further to make use of a combination of file components and registry entries to create a shell on the victim computer. It did this by creating a batch file, a shortcut file, and a random file with a randomised file extension. Registry entries are then created for the randomised file extension to tell the system how to open the file, which contained the malicious code obfuscated with non-ASCII characters. This meant that when the system was restarted or one of the files was triggered, it would run the malicious code and encrypt the victims hard drive.

WannaCry got massive media attention, however, the *Center for Internet Security* reported that in June of 2017, *Kovter* accounted for 44% of malware notifications, and even in March of 2019 – 4 years after its initial creation – still accounted for 14%, 2nd only to *WannaCry*. This can be seen in *Appendices – Center for Internet Security Top 10 Malware.*

Environment Checks

It also implemented many environment detection methods in order to reinforce its efforts to remain undetected. Like *WannaCry, Kovter* checks the computer name, tries to sleep the process, checks system memory and storage, checks running processes, and uses *IsDebuggerPresent*, in order to evade detection.

On top of these measures, it also checks the information of network adapters which can be used to identify virtual machines using MAC addresses and common adapter names such as "*vbox1*" or "*VMNet1*".

Checks adapter addresses which can be used to detect virtual network interfaces (1 event)

Time & API	Arguments
GetAdaptersAddresses April 30, 2019, 9:51 p.m.	flags: 0 family: 0

Figure 14 – Kovter checking for suspicious network adapter settings

It also attempts to fingerprint the machine by checking the BIOS of the system, the machine GUID *(globally unique identifier)*, the Windows Product ID and installation date, and the system uptime. By checking the BIOS, uptime, and installation date, the malware can attempt to identify if it is running in a virtual machine which will usually only be created or booted to analyse the malware.

Collects information to fingerprint the system (MachineGuid, DigitalProductId, SystemBiosDate) (3 events)

registry	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Cryptography\MachineGuid
registry	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\DigitalProductId
registry	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\InstallDate

Looks for the Windows Idle Time to determine the uptime (1 event)

Time & API	Arguments
NtQuerySystemInformation April 30, 2019, 9:49 p.m.	information_class: 8 (SystemProcessorPerformanceInformation)

Figure 15 – Kovter attempting to fingerprint the system

Finally, *Kovter* attempts to check the installed programs for any anti-virus or sniffer programs, such as "*WireShark*" and "*HTTPAnalyzer*". This means that even if it was not in a sandbox, it would evade detection by attempting to disable or manipulate these programs.

Attempts to identify installed AV products by installation directory (1 event)	
file	C:\ProgramData\Trend Micro

Tries to locate whether any sniffers are installed (22 events)	
registry	HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\App Paths\Wireshark.exe
registry	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\App Paths\Wireshark.exe
registry	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Wireshark
registry	HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Wireshark
registry	HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\App Paths\Fiddler.exe
registry	HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\App Paths\Fiddler.exe
registry	HKEY_CURRENT_USER\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Fiddler2

Figure 16 – Kovter checking for installed software that may detect it

ZeuS

"*ZeuS*" is a malware development toolkit used to generate a form of malware known as a banking trojan, first appearing in 2007. [20] A trojan is a malicious executable that is downloaded by piggybacking other downloads, usually hidden inside what appears as a normal looking file – named after the famous story of the Greek forces riding into the city of *Troy* in a giant wooden trojan horse. It used this cover to infiltrate a computer and steal sensitive data such as login credentials, banking details, and other sensitive data, and uploaded them to a remote "*Command and Control*" server hosted by the attacker.

The generated trojans did this by exploiting several vulnerabilities in *ActiveX, JavaScript,* and a host of other software depending on the variation – or by intercepting network traffic. There are many different variations of the "*ZeuS*" toolkit trojans due to the author selling the toolkit on the black market to allow malicious hackers to create their own strains and releasing the original source code in 2011. [21]

One variation of the toolkit was known as "*Citadel*", first appearing in 2012 and was sold for around $3000 on the black market. [22] The Trojans generated by this variant are particularly interesting due to their unique sandbox evasion techniques. When the malware is first downloaded onto the victim machine, it will be placed into a randomised file path with a randomised name, which it then combines with a binary key, encrypts and embeds this fingerprint within the application. Whenever the malware is triggered by being executed or when the victim computer reboots, it generates a new fingerprint based on the same algorithm and checks if they are equal before executing. [23]

Environment Checks

The sample analysed here was a 2013 variant found in *theZoo's* repository as the "*2013BankingVersion*", as strains generated by the "*Citadel*" toolkit are still difficult to find openly on the internet. This variant, unfortunately, does not include the fingerprinting checks but does include some other interesting techniques such as injecting itself into system threads to avoid detection in a sandbox, as seen below.

Used NtSetContextThread to modify a thread in a remote process indicative of process injection (2 events)

Time & API	Arguments	Status	Return	Repeated
Process injection	Process 576 called NtSetContextThread to modify thread in remote process 2020			
NtSetContextThread May 3, 2019, 1:21 p.m.	registers.eip: 2003499180 registers.esp: 3076096 registers.edi: 0 registers.eax: 5 registers.ebp: 0 registers.edx: 10 registers.ebx: 0 registers.esi: 0 registers.ecx: 0 thread_handle: 0x00000094 process_identifier: 2020	1	0	0

Resumed a suspended thread in a remote process potentially indicative of process injection (2 events)

Time & API	Arguments	Status	Return	Repeated
Process injection	Process 576 resumed a thread in remote process 2020			
NtResumeThread May 3, 2019, 1:21 p.m.	thread_handle: 0x00000094 suspend_count: 1 process_identifier: 2020	1	0	0

Figure 17 – ZeuS Trojan injecting itself into another process

Like the *Kovter* example, this ZeuS variant also checks for installed anti-viruses that might detect it to disable them, so the user is not aware the malware has installed.As can be seen

Attempts to identify installed AV products by installation directory (1 event)

file	C:\Program Files\Microsoft Security Client

Figure 18 – ZeuS Trojan trying to identify an install of *Windows Defender*

in the static analysis tab, the malware also tries to bypass detection by using functions which Cuckoo does not flag as suspicious, including checking the clipboard of the computer using "*GetClipboardData*" in "*user32.dll*", checking the uptime of the system using "*GetTickCount*", and checking the *Original Equipment Manufacturer (OEM)* of the system by calling the "*GetOEMCP*" function. Typically, the OEM of a virtual machine will be set to the virtualisation software, and they will likely not contain any clipboard data which can be used to identify if the computer is a sandbox.

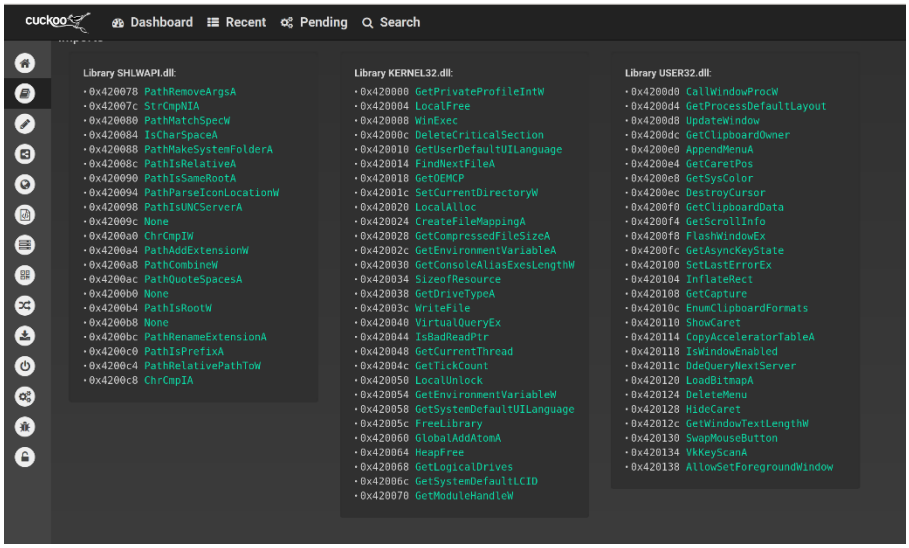


Figure 19 – All library calls made by the ZeuS variant

Discussion

Most malware samples that are successful at infecting computers use varying combinations of environment checks to evade detection. The best way to protect against malware is simply to keep software up-to-date, as most malware will target vulnerabilities that have already been disclosed and patched, meaning the vulnerabilities they utilise only exist on outdated systems. This reduces the attack surface as there are far fewer vulnerabilities which can be exploited by malware to get onto systems and spread.

As technology develops and the use of cloud services increases, organisations have the opportunity to implement alternate methods of securing their end-user device environments.

Hardening Sandboxes

The first is to harden the sandbox used to analyse the malware, making it appear as a standard computer. The flaw with this method is that it is reactive – meaning that it may already be too late to stop malware entering a system or network if it uses cutting-edge techniques to evade the sandbox. This solution requires a lot of maintenance in hardening the sandbox initially and continually updating it to keep up with the latest environment detection techniques.

In the case of *Cuckoo,* which was used to analyse the malware samples in this report, there are not many simple, up-to-date resources on doing this – and many of the tutorials are advanced and require an understanding of *API Hooks* and reverse engineering. [25]

Virtualisation of Desktop Environments

Another solution would be to virtualise desktops. With the presence of cloud computing becoming ever more prevalent in the industry, more companies are moving away from having hundreds or thousands of individual desktops in favour cloud-based tools and running end-point systems off a collection of powerful servers by virtualising desktop environments. This could potentially stop the malware from running as it would potentially detect the virtual environment as a sandbox environment.

Virtualisation can allow clients and employees to access their "*desktop"* from anywhere, on any device with an internet connection using remote desktop protocols. Spreading the virtual machines across a collection of servers massively increases the redundancy of these systems while making maintenance on the virtual environments much easier. Most importantly, however, this would allow a company to easily keep their infrastructure secure by replicating a trusted, up-to-date device image quickly across hundreds or thousands of devices.

Of course, this solution does also bring many security concerns to the table. For instance, the connection would have to be encrypted to prevent man-in-the-middle attacks as these people may be viewing and editing company classified files. For companies implementing these solutions, it is a good idea to keep important files on a separate network, or a computer which can only be accessed physically.

Another concern would be the use of personal devices as there is no guarantee how secure a personal device may be. The device may be run vulnerable software, may not be encrypted, or may have no lock-screen security – meaning a would-be thief would have access to company files.

Cloud-Based User Environments

Many organisations have opted to move their previous in-house infrastructure to the cloud. As most malware will gain access to a company's infrastructure through end-point devices, combining virtualised desktops with cloud-based solutions can help reduce the impact of a cyber-attack. Companies have 2 main choices to enable access to these services and maintain security.

Devices such as Googles "*Chromebooks*", which are built to utilise cloud services. This means very little software is installed locally, and the software that is installed locally is always run within its own sandbox. [26]

Another example of a secure mobile solution would be Apple's "*iPads*". Unless jailbroken, these devices can only download applications from Apple's certified "*App Store*".

Future Work

Initially during the project there were issues getting *Cuckoo* set up. This meant that there was not enough time to use *Ghidra* to reverse engineer and analyse each sample.

Future work would be analysing the samples by reverse engineering them in *Ghidra* to identify exactly what functions are carried out using the information gathered about the environment before executing their payload.

There were samples that would not run in *Cuckoo* due to them detecting the sandbox environment. Further research into hardening the sandbox against detection by malware using tools such as *Pafish* – which runs a variety of common environment checks and outputs which ones it can successfully detect. [7]

References

1. *7 cybersecurity trends to watch out for in 2018*; by Aidan Simister. Available From: https://www.csoonline.com/article/3250086/7-cybersecurity-trends-to-watch-out-for-in-2018.html [Accessed 18/04/2019]
2. *W32.Stuxnet Dossier*; by Nicolas Falliere, Liam O Murchu, and Eric Chein. Available From: https://www.symantec.com/content/en/us/enterprise/media/security_response/whitepapers/w32_stuxnet_dossier.pdf [Accessed 02/05/2019]
3. *5 Ways Advanced Malware Evades the Sandbox.* Available From: https://www.secureworks.com/blog/bg-5-ways-malware-evades-the-sandbox [Accessed 18/04/2019]
4. *Sandbox Evasion Techniques – Part 4.* Available From: https://www.vmray.com/cyber-security-blog/sandbox-evasion-techniques-part-4/ [Accessed 19/04/2019]
5. *IsDebuggerPresent Documentation.* Available From: https://www.aldeid.com/wiki/IsDebuggerPresent [Accessed 19/04/2019]
6. *Desktop Apps: How to deal with malware that uses "Anti-Debugging" techniques*. Available From: https://social.technet.microsoft.com/wiki/contents/articles/52123.desktop-apps-how-to-deal-with-malware-that-uses-anti-debugging-techniques.aspx [Accessed 19/04/2019]
7. *VM Detection – passing the Pafish test.* Available From: https://www.vmray.com/cyber-security-blog/a-pafish-primer/ [Accessed 19/04/2019]
8. *theZoo.* Available From: https://github.com/ytisf/theZoo [Accessed 02/05/2019]
9. *Making a portable full installation of Ubuntu on a USB HDD*. Available From: https://www.dionysopoulos.me/portable-ubuntu-on-usb-hdd/ [Accessed 12/03/2019]
10. *Can files locked by WannaCry be decrypted: A technical analysis*. Available From: https://medium.com/threat-intel/wannacry-ransomware-decryption-821c7e3f0a2b [Accessed 02/05/2019]
11. *WannaCry cyber attack cost the NHS £92m as 19,000 appointments cancelled*. Available From: https://www.telegraph.co.uk/technology/2018/10/11/wannacry-cyber-attack-cost-nhs-92m-19000-appointments-cancelled/ [Accessed 02/05/2019]
12. *Everything you need to know about EternalBlue – the NSA exploit linked to Petya*. Available From: https://www.wired.co.uk/article/what-is-eternal-blue-exploit-vulnerability-patch [Accessed 02/05/2019]
13. *What is WannaCry ransomware, how does it infect, and who was responsible?*. Available From: https://www.csoonline.com/article/3227906/what-is-wannacry-ransomware-how-does-it-infect-and-who-was-responsible.html [Accessed 02/05/2019]

14. *HOW AN ACCIDENTAL 'KILL SWITCH' SLOWED FRIDAY'S MASSIVE RANSOMWARE ATTACK*. Available From: https://www.wired.com/2017/05/accidental-kill-switch-slowed-fridays-massive-ransomware-attack/ [Accessed 02/05/2019]
15. *Reverse Engineering Wannacry*. Available From: https://www.ghidra.ninja/posts/03-wannacry-1/ [Accessed 03/05/2019]
16. *WinMain: The Application Entry Point*. Available From: https://docs.microsoft.com/en-us/windows/desktop/learnwin32/winmain--the-application-entry-point [Accessed 03/05/2019]
17. *InternetOpenUrlA function*. Available From: https://docs.microsoft.com/en-us/windows/desktop/api/wininet/nf-wininet-internetopenurla [Accessed 03/05/2019]
18. *KOVTER: An Evolving Malware Gone Fileless*. Available From: https://www.trendmicro.com/vinfo/us/security/news/cybercrime-and-digital-threats/kovter-an-evolving-malware-gone-fileless [Accessed 19/04/2019]
19. *Fileless infections: an overview*. Available From: https://blog.malwarebytes.com/cybercrime/2016/03/fileless-infections-an-overview/ [Accessed 02/05/2019]
20. *Zeus Virus*. Available From: https://usa.kaspersky.com/resource-center/threats/zeus-virus [Accessed 03/05/2019]
21. *Zeus is Still the Base of Many Current Trojans*. Available From: https://www.pandasecurity.com/mediacenter/panda-security/zeus-trojan/ [Accessed 03/05/2019]
22. *Citadel: a cyber-criminal's ultimate weapon?* Available From: https://blog.malwarebytes.com/threat-analysis/2012/11/citadel-a-cyber-criminals-ultimate-weapon/ [Accessed 03/05/2019]
23. *Analyzing Environment-Aware Malware*. Available From: https://www.lastline.com/labsblog/analyzing-environment-aware-malware/ [Accessed 03/05/2019]
24. *What are cloud desktops and how can they benefit my business?* Available From: https://www.itpro.co.uk/virtualisation/32245/what-are-cloud-desktops-and-how-can-they-benefit-my-business [Accessed 04/05/2019]
25. *Prevalent Threats Targeting Cuckoo Sandbox Detection and Our Mitigation*. Available From: https://www.fortinet.com/blog/threat-research/prevalent-threats-targeting-cuckoo-sandbox-detection-and-our-mitigation.html [Accessed 04/05/2019]
26. *ChromeBook Help Documentation – ChromeBook Security*. Available From: https://support.google.com/chromebook/answer/3438631?hl=en-GB [Accessed 04/05/2019]

Appendices

"cuckoo-install.sh" Configuration

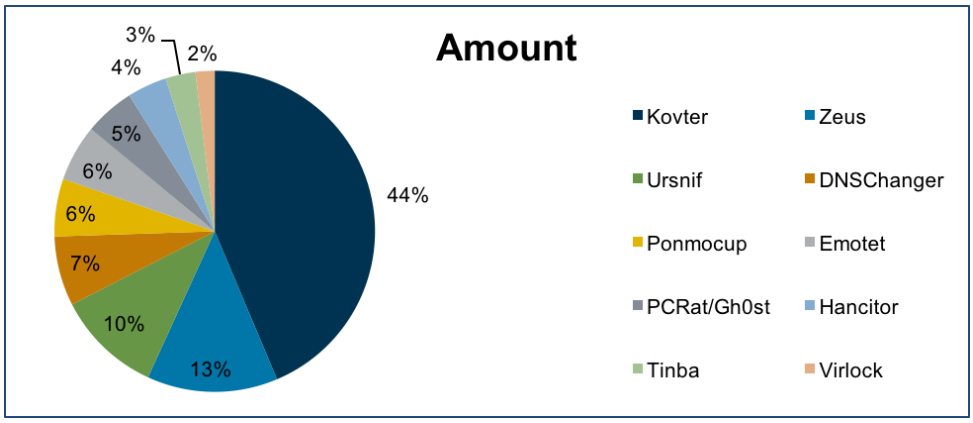
```
CUCKOO_GUEST_IMAGE="/cuckoo/WIN7.ovf"
CUCKOO_GUEST_NAME="cuckoo1"
CUCKOO_GUEST_IP="192.168.56.15"
INTERNET_INT_NAME="wlp1s0"
```

Network Configuration Commands

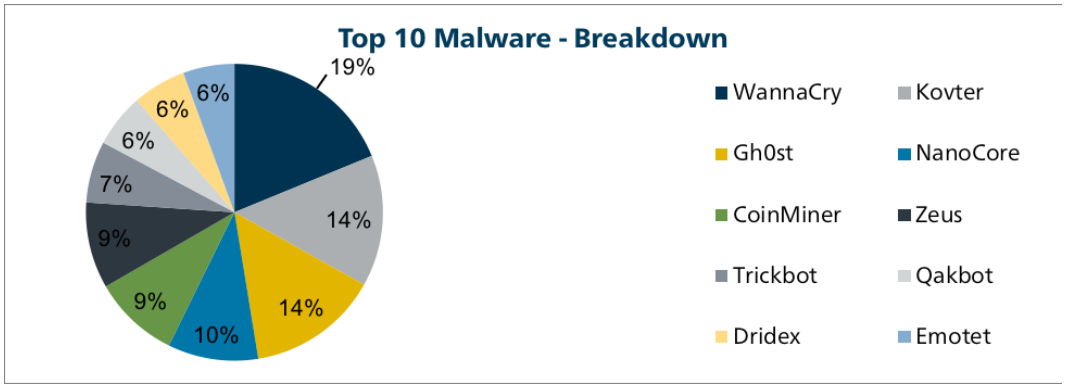
```
sudo apt-get install iptables-save -y
sudo apt-get install iptables-persistent -y
sudo iptables -A INPUT -i lo -j ACCEPT
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A FORWARD -s 192.168.56.0/24 -d 192.168.0.0/16 -i vboxnet1 -
o wlp1s0 -j DROP
sudo iptables -A FORWARD -s 192.168.56.0/24 -d 172.16.0.0/12 -i vboxnet1 -o
wlp1s0 -j DROP
sudo iptables -A FORWARD -s 192.168.56.0/24 -d 10.0.0.0/8 -i vboxnet1 -o
wlp1s0 -j DROP
sudo iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 22 -j
ACCEPT
sudo iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 5900 -j
ACCEPT
sudo iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 5901 -j
ACCEPT
sudo iptables -A INPUT -m state --state NEW -m tcp -p tcp --dport 8000 -j
ACCEPT
sudo iptables -P INPUT DROP
sudo iptables-save > /etc/iptables/rules.v4
sudo gsettings set org.gnome.Vino require-encryption false
```

Center for Internet Security Top 10 Malware

June 2017 (https://www.cisecurity.org/blog/top-10-malware-of-june-2017/)



March 2019 (https://www.cisecurity.org/blog/top-10-malware-march-2019/)



WannaCry Kill-Switch

Decompile: WinMain - (wannacry)

```
1
2  int WinMain(HINSTANCE hInstance,HINSTANCE hPrevInstance,PWSTR pCmdLine,int nCmdShow)
3
4  {
5    HINTERNET hInternet;
6    HINTERNET hInternetReturn;
7    int i;
8    char *URL1;
9    char *url_copy;
10   char url_buffer [57];
11
12   i = 14;
13   URL1 = s_http://www.iuqerfsodp9ifjaposdfj_004313d0;
14   url_copy = url_buffer;
15   while (i != 0
16                    /* strcopy(URL_copy, URL1, 14) */) {
17     i = i + -1;
18     *(undefined4 *)url_copy = *(undefined4 *)URL1;
19     URL1 = URL1 + 4;
20     url_copy = url_copy + 4;
21   }
22   *url_copy = *URL1;
23   InternetOpenA((LPCSTR)0x0,1,(LPCSTR)0x0,(LPCSTR)0x0,0);
24   hInternetReturn = InternetOpenUrlA(hInternet,url_buffer,(LPCSTR)0x0,0,0x84000000,0);
25   if (hInternetReturn == (HINTERNET)0x0) {
26     InternetCloseHandle(hInternet);
27     InternetCloseHandle(0);
28     real_entry();
29     return 0;
30   }
31                    /* Tries to request the URL. If it fails, it closes the handler & calls a
32                       function. If it succeeds, it closes the handler and returns 0, closing the
33                       program */
34   InternetCloseHandle(hInternet);
35   InternetCloseHandle(hInternetReturn);
36   return 0;
37 }
38
```